



RTL Design Sherpa

RTL AMBA AXI5-Lite Module Reference — Rev 0.90

October 2026

Table of Contents

1 AXI5-Lite (AXIL5) Modules.....	14
1.1 Overview.....	14
1.2 Functional Description.....	14
1.3 Usage Example.....	15
1.4 Design Notes.....	16
1.5 Related Modules.....	16
1.5.1 Transport (4).....	16
1.5.2 Clock-gated (4).....	16
1.5.3 Monitored (4).....	17
1.5.4 Monitored and clock-gated (4).....	17
1.6 Testing.....	17
1.7 Navigation.....	17
2 AXIL5 Master Read.....	18
2.1 Overview.....	18
2.2 Parameters.....	18
2.3 Ports.....	19
2.4 Functional Description.....	21
2.4.1 Optional signal groups.....	21
2.4.2 With every group disabled, this IS the AXI4-Lite module.....	22
2.4.3 It transports; it does not interpret.....	22
2.5 Timing Characteristics.....	22
2.5.1 Buffer Depths and Latency.....	22

2.5.2 Optional-group effect.....	23
2.6 Usage Examples.....	23
2.7 Design Notes.....	24
2.8 Related Modules.....	24
2.9 Testing.....	25
2.10 Navigation.....	25
3 AXIL5 Master Read, Clock-Gated.....	26
3.1 Overview.....	26
3.2 Parameters.....	26
3.3 Ports.....	27
3.4 Functional Description.....	30
3.4.1 Optional signal groups.....	30
3.4.2 With every group disabled, this IS the AXI4-Lite module.....	31
3.4.3 It transports; it does not interpret.....	31
3.5 Timing Characteristics.....	32
3.5.1 Buffer Depths and Latency.....	32
3.5.2 Optional-group effect.....	32
3.6 Usage Examples.....	32
3.7 Design Notes.....	33
3.8 Related Modules.....	34
3.9 Testing.....	34
3.10 Navigation.....	35
4 AXIL5 Master Read Monitor.....	36
4.1 Overview.....	36

4.2 Parameters.....	36
4.3 Ports.....	40
4.4 Functional Description.....	45
4.4.1 Optional signal groups.....	45
4.4.2 With every group disabled, this IS the AXI4-Lite module.....	46
4.4.3 It transports; it does not interpret.....	46
4.5 Design Notes.....	46
4.6 Timing Characteristics.....	47
4.6.1 Buffer Depths and Latency.....	47
4.6.2 Optional-group effect.....	47
4.7 Usage Examples.....	48
4.8 Related Modules.....	49
4.9 Testing.....	49
4.10 Navigation.....	50
5 AXI5 Master Read Monitor, Clock-Gated.....	51
5.1 Overview.....	51
5.2 Parameters.....	51
5.3 Ports.....	54
5.4 Functional Description.....	59
5.4.1 Optional signal groups.....	59
5.4.2 With every group disabled, this IS the AXI4-Lite module.....	60
5.4.3 It transports; it does not interpret.....	60
5.5 Design Notes.....	61
5.6 Timing Characteristics.....	61

5.6.1 Buffer Depths and Latency.....	61
5.6.2 Optional-group effect.....	62
5.7 Usage Examples.....	62
5.8 Related Modules.....	63
5.9 Testing.....	64
5.10 Navigation.....	64
6 AXIL5 Master Write.....	65
6.1 Overview.....	65
6.2 Parameters.....	65
6.3 Ports.....	66
6.4 Functional Description.....	68
6.4.1 Optional signal groups.....	68
6.4.2 With every group disabled, this IS the AXI4-Lite module.....	69
6.4.3 It transports; it does not interpret.....	69
6.5 Timing Characteristics.....	70
6.5.1 Buffer Depths and Latency.....	70
6.5.2 Optional-group effect.....	70
6.6 Usage Examples.....	70
6.7 Design Notes.....	72
6.8 Related Modules.....	72
6.9 Testing.....	72
6.10 Navigation.....	73
7 AXIL5 Master Write, Clock-Gated.....	74
7.1 Overview.....	74

7.2 Parameters.....	74
7.3 Ports.....	76
7.4 Functional Description.....	78
7.4.1 Optional signal groups.....	78
7.4.2 With every group disabled, this IS the AXI4-Lite module.....	78
7.4.3 It transports; it does not interpret.....	79
7.5 Timing Characteristics.....	79
7.5.1 Buffer Depths and Latency.....	79
7.5.2 Optional-group effect.....	80
7.6 Usage Examples.....	80
7.7 Design Notes.....	81
7.8 Related Modules.....	82
7.9 Testing.....	82
7.10 Navigation.....	82
8 AXIL5 Master Write Monitor.....	83
8.1 Overview.....	83
8.2 Parameters.....	83
8.3 Ports.....	87
8.4 Functional Description.....	92
8.4.1 Optional signal groups.....	92
8.4.2 With every group disabled, this IS the AXI4-Lite module.....	93
8.4.3 It transports; it does not interpret.....	93
8.5 Design Notes.....	94
8.6 Timing Characteristics.....	94

8.6.1 Buffer Depths and Latency.....	94
8.6.2 Optional-group effect.....	95
8.7 Usage Examples.....	95
8.8 Related Modules.....	96
8.9 Testing.....	97
8.10 Navigation.....	97
9 AXIL5 Master Write Monitor, Clock-Gated.....	98
9.1 Overview.....	98
9.2 Parameters.....	98
9.3 Ports.....	101
9.4 Functional Description.....	107
9.4.1 Optional signal groups.....	107
9.4.2 With every group disabled, this IS the AXI4-Lite module.....	107
9.4.3 It transports; it does not interpret.....	108
9.5 Design Notes.....	108
9.6 Timing Characteristics.....	109
9.6.1 Buffer Depths and Latency.....	109
9.6.2 Optional-group effect.....	109
9.7 Usage Examples.....	109
9.8 Related Modules.....	111
9.9 Testing.....	111
9.10 Navigation.....	111
10 AXIL5 Slave Read.....	112
10.1 Overview.....	112

10.2 Parameters.....	112
10.3 Ports.....	113
10.4 Functional Description.....	115
10.4.1 Optional signal groups.....	115
10.4.2 With every group disabled, this IS the AXI4-Lite module.....	116
10.4.3 It transports; it does not interpret.....	116
10.5 Timing Characteristics.....	116
10.5.1 Buffer Depths and Latency.....	116
10.5.2 Optional-group effect.....	117
10.6 Usage Examples.....	117
10.7 Design Notes.....	118
10.8 Related Modules.....	118
10.9 Testing.....	119
10.10 Navigation.....	119
11 AXIL5 Slave Read, Clock-Gated.....	120
11.1 Overview.....	120
11.2 Parameters.....	120
11.3 Ports.....	121
11.4 Functional Description.....	123
11.4.1 Optional signal groups.....	123
11.4.2 With every group disabled, this IS the AXI4-Lite module.....	124
11.4.3 It transports; it does not interpret.....	124
11.5 Timing Characteristics.....	125
11.5.1 Buffer Depths and Latency.....	125

11.5.2 Optional-group effect.....	125
11.6 Usage Examples.....	125
11.7 Design Notes.....	126
11.8 Related Modules.....	127
11.9 Testing.....	127
11.10 Navigation.....	128
12 AXIL5 Slave Read Monitor.....	129
12.1 Overview.....	129
12.2 Parameters.....	129
12.3 Ports.....	133
12.4 Functional Description.....	138
12.4.1 Optional signal groups.....	138
12.4.2 With every group disabled, this IS the AXI4-Lite module.....	139
12.4.3 It transports; it does not interpret.....	139
12.5 Design Notes.....	139
12.6 Timing Characteristics.....	140
12.6.1 Buffer Depths and Latency.....	140
12.6.2 Optional-group effect.....	140
12.7 Usage Examples.....	141
12.8 Related Modules.....	142
12.9 Testing.....	142
12.10 Navigation.....	143
13 AXIL5 Slave Read Monitor, Clock-Gated.....	144
13.1 Overview.....	144

13.2 Parameters.....	144
13.3 Ports.....	147
13.4 Functional Description.....	152
13.4.1 Optional signal groups.....	152
13.4.2 With every group disabled, this IS the AXI4-Lite module.....	153
13.4.3 It transports; it does not interpret.....	153
13.5 Design Notes.....	154
13.6 Timing Characteristics.....	154
13.6.1 Buffer Depths and Latency.....	154
13.6.2 Optional-group effect.....	155
13.7 Usage Examples.....	155
13.8 Related Modules.....	156
13.9 Testing.....	157
13.10 Navigation.....	157
14 AXI5 Slave Write.....	158
14.1 Overview.....	158
14.2 Parameters.....	158
14.3 Ports.....	159
14.4 Functional Description.....	161
14.4.1 Optional signal groups.....	161
14.4.2 With every group disabled, this IS the AXI4-Lite module.....	162
14.4.3 It transports; it does not interpret.....	162
14.5 Timing Characteristics.....	163
14.5.1 Buffer Depths and Latency.....	163

14.5.2 Optional-group effect.....	163
14.6 Usage Examples.....	163
14.7 Design Notes.....	165
14.8 Related Modules.....	165
14.9 Testing.....	165
14.10 Navigation.....	166
15 AXIL5 Slave Write, Clock-Gated.....	167
15.1 Overview.....	167
15.2 Parameters.....	167
15.3 Ports.....	169
15.4 Functional Description.....	171
15.4.1 Optional signal groups.....	171
15.4.2 With every group disabled, this IS the AXI4-Lite module.....	171
15.4.3 It transports; it does not interpret.....	172
15.5 Timing Characteristics.....	172
15.5.1 Buffer Depths and Latency.....	172
15.5.2 Optional-group effect.....	173
15.6 Usage Examples.....	173
15.7 Design Notes.....	174
15.8 Related Modules.....	175
15.9 Testing.....	175
15.10 Navigation.....	175
16 AXIL5 Slave Write Monitor.....	176
16.1 Overview.....	176

16.1.1 Key Features.....	176
16.2 Parameters.....	176
16.3 Ports.....	180
16.4 Functional Description.....	185
16.4.1 With every group disabled, this IS the AXI4-Lite module.....	186
16.4.2 It transports; it does not interpret.....	186
16.5 Design Notes.....	187
16.6 Timing Characteristics.....	187
16.6.1 Buffer Depths and Latency.....	187
16.6.2 Optional-group effect.....	188
16.7 Usage Examples.....	188
16.8 Related Modules.....	189
16.9 Testing.....	190
16.10 Navigation.....	190
17 AXIL5 Slave Write Monitor, Clock-Gated.....	191
17.1 Overview.....	191
17.1.1 Key Features.....	191
17.2 Parameters.....	191
17.3 Ports.....	194
17.4 Functional Description.....	200
17.4.1 With every group disabled, this IS the AXI4-Lite module.....	200
17.4.2 It transports; it does not interpret.....	201
17.5 Design Notes.....	201
17.6 Timing Characteristics.....	202

17.6.1 Buffer Depths and Latency.....	202
17.6.2 Optional-group effect.....	202
17.7 Usage Examples.....	202
17.8 Related Modules.....	204
17.9 Testing.....	204
17.10 Navigation.....	204

1 AXI5-Lite (AXIL5) Modules

Location: rtl/amba/axil5/ **Sibling family:** [AXI4-Lite](#)

1.1 Overview

Sixteen modules mirroring the AXI4-Lite family one for one, with the AXI5-Lite optional signal groups threaded through. Same channel set, same handshakes, same single-beat semantics – what AXI5-Lite adds is sideband.

If you came here looking for a full AXI5-Lite protocol stack, read the next section before you instantiate anything.

1.2 Functional Description

These modules **transport** AXI5-Lite signals. They do not execute AXI5-Lite semantics:

- MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. Nothing here interprets a partition ID or an encryption context.
- POISON is carried. It is never generated and never checked.
- LOCK is carried with no exclusive-access monitor behind it. A completer returning EXOKAY is not validated against anything.

Those behaviours belong to the endpoints on either side. Read this before treating the family as a full AXI5-Lite protocol stack – it is not one.

AXI4-Lite vs AXI5-Lite, as this library implements them:

Feature	AXI4-Lite	AXI5-Lite
Channels	AW, W, B, AR, R	same
Burst support	none	none
Transaction ID	none	none
User sideband	not present	AxUSER/WUSER/BUSER/ RUSER, gated by ENABLE_USER
Trace	not present	AxTRACE/BTRACE/RTRACE,

Feature	AXI4-Lite	AXI5-Lite
Loopback ID	not present	gated by ENABLE_TRACE AxLOOP/BLLOOP/RLLOOP, gated by ENABLE_LOOP
MPAM / MECID / NSAID	not present	on the address channels, each independently gated
Poison	not present	WPOISON/RPOISON, one bit per 64 data bits
Exclusive access	not present	AxLOCK, gated by ENABLE_LOCK

With every group disabled the two are the same interface. The packed payload widths match channel for channel, which is what lets one testbench bind to either family.

1.3 Usage Example

Two configurations of the same module – every group off, and the security qualifiers on:

```
// AXI4-Lite-equivalent: every group off, payload widths identical to
axil4
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .ENABLE_USER (0), .ENABLE_TRACE (0), .ENABLE_LOOP
(0), .ENABLE_MPAM (0),
    .ENABLE_MECID(0), .ENABLE_NSAID(0), .ENABLE_POISON(0), .ENABLE_LOCK (0)
) u_plain ( /* ... */ );

// With the security qualifiers, which is what AXI5-Lite is usually
for
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .NSAID_WIDTH (4), .MPAM_WIDTH (11), .MECID_WIDTH (16),
    .ENABLE_NSAID(1), .ENABLE_MPAM(1), .ENABLE_MECID(1),
    .ENABLE_USER (0), .ENABLE_TRACE(0), .ENABLE_LOOP (0),
    .ENABLE_POISON(0), .ENABLE_LOCK(0)
) u_secure ( /* ... */ );
```

1.4 Design Notes

Known limitations:

- No protocol checking of the optional groups. The monitored variants observe handshakes, addresses, responses and timing; axi_monitor_filtered has no ports for MPAM, MECID, NSCID, TRACE, LOOP or POISON.
 - No exclusive-access monitor. AxLOCK is transported; nothing tracks exclusive state or validates EXOKAY.
 - No poison generation or checking.
 - Timing diagrams are not yet drawn for this family.
-

1.5 Related Modules

The family, grouped by what each variant adds:

1.5.1 Transport (4)

Module	Channels
axil5_master_rd	AR, R
axil5_master_wr	AW, W, B
axil5_slave_rd	AR, R
axil5_slave_wr	AW, W, B

1.5.2 Clock-gated (4)

Module	Adds
axil5_master_rd_cg	One amba_clock_gate_ctrl over the whole inner module
axil5_master_wr_cg	One amba_clock_gate_ctrl over the whole inner module
axil5_slave_rd_cg	One amba_clock_gate_ctrl over the whole inner module
axil5_slave_wr_cg	One amba_clock_gate_ctrl over the whole inner module

1.5.3 Monitored (4)

Module	Adds
axil5_master_rd_mon	axi_monitor_filtered, monbus packet output
axil5_master_wr_mon	axi_monitor_filtered, monbus packet output
axil5_slave_rd_mon	axi_monitor_filtered, monbus packet output
axil5_slave_wr_mon	axi_monitor_filtered, monbus packet output

1.5.4 Monitored and clock-gated (4)

Module	Adds
axil5_master_rd_mon_cg	Both, with ready held low while gated
axil5_master_wr_mon_cg	Both, with ready held low while gated
axil5_slave_rd_mon_cg	Both, with ready held low while gated
axil5_slave_wr_mon_cg	Both, with ready held low while gated

1.6 Testing

`val/amba/test_axil5_*.py` – sixteen runners, 42 parameterised cases, all passing. They drive the modules with **every optional group enabled**, which makes them the only tests in the repo that exercise the AXI5-Lite sideband against real transport RTL rather than a behavioural model.

The testbench classes in `bin/TBClasses/axil5/` are the AXI4-Lite ones with the component factories swapped and the group widths set in `COMPONENT_KWARGS`. One definition of the traffic and its checks.

Also here: `test_axil5_opt_signals.py` drives `axil5_opt_slave`, a behavioural slave under `rtl/amba/axil5/test-modules/` built purely so the BFM's had a DUT carrying every optional port. It is test collateral – do not instantiate it in a design.

1.7 Navigation

[AMBA overview](#) | [AXI4-Lite](#) | [AXI5](#)

2 AXIL5 Master Read

Module: axil5_master_rd.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_master_rd

2.1 Overview

The AXIL5 Master Read provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_master_rd with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
-

2.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	$(DW / 64) > 0$? $(DW / 64) :$ 1	
ARSize	int	$AW + 3 +$	
RSize	int	$DW + 2 +$	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

2.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
fub_araddr	Input	[AW-1:0]	

Port	Direction	Width	Description
fub_arprot	Input	[2:0]	
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_arpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	

Port	Direction	Width	Description
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
busy	Output	1	

2.4 Functional Description

2.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

2.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

2.4.3 It transports; it does not interpret

MPAM, MECID, NSCID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

2.5 Timing Characteristics

2.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

2.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

2.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AR        (2),
    .SKID_DEPTH_R         (4)
) u_axil5_master_rd (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // fub_ar
    .fub_araddr           (fub_araddr),
```

```

// ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
// m_axil_ar
.m_axil_araddr      (m_axil_araddr),
// ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
// m_axil_r
.m_axil_rdata       (m_axil_rdata),
// ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
// fub_r
.fub_rdata          (fub_rdata),
// ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
// status
.busy                (busy)
);

```

2.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM's against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

2.8 Related Modules

- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_rd_mon_cg](#)
- [axil5_master_wr](#)
- [axil5_master_wr_cg](#)
- [axil5_master_wr_mon](#)

- `axil4_master_rd` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

2.9 Testing

`val/amba/test_axil5_master_rd.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

2.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

3 AXIL5 Master Read, Clock-Gated

Module: axil5_master_rd_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_master_rd_cg

3.1 Overview

The AXIL5 Master Read, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_master_rd_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	

Parameter	Type	Default	Description
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

3.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_araddr	Input	[AW-1:0]	
fub_arprot	Input	[2:0]	
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_arpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	

Port	Direction	Width	Description
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

3.4 Functional Description

3.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1

Group	Parameter	Signals on this module	Width
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

3.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

3.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

3.5 Timing Characteristics

3.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

3.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

3.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
```

```

        .ENABLE_LOOP            (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID          (1),
        .ENABLE_NSAID          (1),
        .ENABLE_POISON         (1),
        .ENABLE_LOCK            (1),
        .SKID_DEPTH_AR         (2),
        .SKID_DEPTH_R          (4),
        .CG_IDLE_COUNT_WIDTH    (4)
    ) u_axil5_master_rd_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // configuration
        .cfg_cg_enable           (cfg_cg_enable),
        .cfg_cg_idle_count       (cfg_cg_idle_count),
        // fub_ar
        .fub_araddr              (fub_araddr),
        // ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
        // m_axil_ar
        .m_axil_araddr           (m_axil_araddr),
        // ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
        // m_axil_r
        .m_axil_rdata            (m_axil_rdata),
        // ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
        // fub_r
        .fub_rdata               (fub_rdata),
        // ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
        // status
        .cg_gating               (cg_gating),
        .cg_idle                 (cg_idle)
    );

```

3.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFMs against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating` – see `vault/handbook/design/clock-gating-activity-terms.md`.

busy is not brought out here. The plain `_cg` wrappers consume the base module's busy internally as a wake term; use `cg_idle` or `cg_gating` for idle detection. The `_mon_cg` variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

3.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - `axil4_master_rd_cg` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

3.9 Testing

`val/amba/test_axil5_master_rd_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

3.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

4 AXIL5 Master Read Monitor

Module: `axil5_master_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon`

4.1 Overview

The AXIL5 Master Read Monitor provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

4.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE

Parameter	Type	Default	Description
			=1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told

Parameter	Type	Default	Description
			apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

4.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_araddr	Input	[AW-1:0]	
fub_axil_arprot	Input	[2:0]	
fub_axil_arlock	Input	1	
fub_axil_aruser	Input	[UW-1:0]	
fub_axil_artrace	Input	1	
fub_axil_arloop	Input	[LW-1:0]	
fub_axil_armpam	Input	[MW-1:0]	
fub_axil_armecid	Input	[EW-1:0]	
fub_axil_arnsaid	Input	[NW-1:0]	
fub_axil_arvalid	Input	1	
fub_axil_arready	Output	1	
fub_axil_rdata	Output	[DW-1:0]	
fub_axil_rresp	Output	[1:0]	

Port	Direction	Width	Description
fub_axil_ruser	Output	[UW-1:0]	
fub_axil_rtrace	Output	1	
fub_axil_rloop	Output	[LW-1:0]	
fub_axil_rpoison	Output	[PW-1:0]	
fub_axil_rvalid	Output	1	
fub_axil_rready	Input	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask

Port	Direction	Width	Description
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the

Port	Direction	Width	Description
			group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force	Input	1	

Port	Direction	Width	Description
_close			
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

4.4 Functional Description

4.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

4.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

4.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

4.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.

axi_monitor_trans_mgr fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets ADDR_FILTER_ENABLE but leaves cfg_addr_filter_enable low filters nothing and looks broken. The parameter and the runtime port are both required.

4.6 Timing Characteristics

4.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

4.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, ASize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

4.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R      (4),
```

```

.AXIL_ADDR_WIDTH      (32),
.AXIL_DATA_WIDTH      (32),
.ACLK_MHZ             (100),
.USE_MONITOR          (1'b1),
.N_ADDR_RANGES        (0),
.UNIT_ID              (8'h01),
.AGENT_ID             (16'h000A),
.MAX_TRANSACTIONS     (8),
.USE_WDATA_ORDER_Q    (1'b0),
.NUM_BANKS            (1),
.ID_FILTER_ENABLE     (1'b0),
.ADDR_FILTER_ENABLE   (1'b0),
.ID_MATCH_BASE        (0),
.ID_MATCH_COUNT       (0),
.ENABLE_FILTERING     (1),
.ADD_PIPELINE_STAGE   (0),
.ENABLE_ERROR_LOGIC   (1'b1),
.ENABLE_TIMEOUT_LOGIC (1'b1),
.ENABLE_COMPL_LOGIC   (1'b1),
.ENABLE_THRESHOLD_LOGIC (1'b1),
.ENABLE_PERF_LOGIC    (1'b1),
.ENABLE_DEBUG_LOGIC   (1'b0),
.USER_WIDTH           (4),
.LOOP_WIDTH           (3),
.MPAM_WIDTH           (11),
.MECID_WIDTH          (16),
.NSAID_WIDTH          (4),
.ENABLE_USER          (1),
.ENABLE_TRACE         (1),
.ENABLE_LOOP          (1),
.ENABLE_MPAM          (1),
.ENABLE_MECID         (1),
.ENABLE_NSAID         (1),
.ENABLE_POISON        (1),
.ENABLE_LOCK          (1)
) u_axil5_master_rd_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
    // m_axil_ar
    .m_axil_araddr       (m_axil_araddr),
    // ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
    // m_axil_r
    .m_axil_rdata        (m_axil_rdata),

```

```
// ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
// configuration
.cfg_monitor_enable (cfg_monitor_enable),
// ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
// monitor bus
.monbus_valid      (monbus_valid)
// ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

4.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_rd_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

4.9 Testing

`val/amba/test_axil5_master_rd_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in COMPONENT_KWARGS to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

4.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

5 AXIL5 Master Read Monitor, Clock-Gated

Module: `axil5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon_cg`

5.1 Overview

The AXIL5 Master Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

5.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_master_rd_monlite_cg`: the same gating logic around `axil5_master_rd_monlite` instead of `axil5_master_rd_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon;

Parameter	Type	Default	Description
N_ADDR_RANGES	int	0	outputs tied 0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	

Parameter	Type	Default	Description
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000A	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier

Parameter	Type	Default	Description
			emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

5.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_araddr	Input	[AW-1:0]	
fub_axil_arprot	Input	[2:0]	
fub_axil_arlock	Input	1	
fub_axil_aruser	Input	[UW-1:0]	
fub_axil_artrace	Input	1	

Port	Direction	Width	Description
fub_axil_arloop	Input	[LW-1:0]	
fub_axil_arpam	Input	[MW-1:0]	
fub_axil_armecid	Input	[EW-1:0]	
fub_axil_arnsaid	Input	[NW-1:0]	
fub_axil_arvalid	Input	1	
fub_axil_arready	Output	1	
fub_axil_rdata	Output	[DW-1:0]	
fub_axil_rresp	Output	[1:0]	
fub_axil_ruser	Output	[UW-1:0]	
fub_axil_rtrace	Output	1	
fub_axil_rloop	Output	[LW-1:0]	
fub_axil_rpoison	Output	[PW-1:0]	
fub_axil_rvalid	Output	1	
fub_axil_rready	Input	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_arpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	

Port	Direction	Width	Description
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error

Port	Direction	Width	Description
			event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1 : 0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready

Port	Direction	Width	Description
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions

Port	Direction	Width	Description
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

5.4 Functional Description

5.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

5.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

5.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

5.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

5.6 Timing Characteristics

5.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

5.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

5.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_rd_mon_cg #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .ADDR_FILTER_ENABLE (1'b0),
    .ACLK_MHZ           (100),
    .USE_WDATA_ORDER_Q  (1'b0),
    .NUM_BANKS          (1),
    .UNIT_ID             (8'h01),
    .AGENT_ID           (16'h000A),
    .MAX_TRANSACTIONS   (8),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE (0),
    .ENABLE_ERROR_LOGIC (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC  (1'b1),
    .ENABLE_DEBUG_LOGIC (1'b0),
    .CG_IDLE_COUNT_WIDTH (4),
    .USER_WIDTH          (4),
    .LOOP_WIDTH          (3),
    .MPAM_WIDTH          (11),
    .MECID_WIDTH         (16),
    .NSAID_WIDTH         (4),
    .ENABLE_USER         (1),
    .ENABLE_TRACE        (1),
    .ENABLE_LOOP         (1),
```

```

        .ENABLE_MPAM            (1),
        .ENABLE_MECID          (1),
        .ENABLE_NSAID          (1),
        .ENABLE_POISON         (1),
        .ENABLE_LOCK            (1)
    ) u_axil5_master_rd_mon_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // status
        .cam_clear               (cam_clear),
        // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
        // m_axil_ar
        .m_axil_araddr           (m_axil_araddr),
        // ... `m_axil_arprot`, `m_axil_arlock`, `m_axil_aruser`, +7 more
        // m_axil_r
        .m_axil_rdata            (m_axil_rdata),
        // ... `m_axil_rresp`, `m_axil_ruser`, `m_axil_rtrace`, +4 more
        // configuration
        .cfg_monitor_enable      (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
        // monitor bus
        .monbus_valid            (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

5.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_rd_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

5.9 Testing

`val/amba/test_axil5_master_rd_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

5.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

6 AXIL5 Master Write

Module: `axil5_master_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr`

6.1 Overview

The AXIL5 Master Write provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
-

6.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	
BSize	int	2 +	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

6.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus.

Port	Direction	Width	Description
aclk	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	
fub_awmecid	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmeci d	Output	[EW-1:0]	
m_axil_awnsai d	Output	[NW-1:0]	
m_axil_awvali d	Output	1	
m_axil_awread y	Input	1	
fub_wdata	Input	[DW-1:0]	
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	

Port	Direction	Width	Description
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
busy	Output	1	

6.4 Functional Description

6.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

6.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

6.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

6.5 Timing Characteristics

6.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

6.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

6.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr #(
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USER_WIDTH          (4),
```

```

        .LOOP_WIDTH           (3),
        .MPAM_WIDTH           (11),
        .MECID_WIDTH           (16),
        .NSAID_WIDTH           (4),
        .ENABLE_USER           (1),
        .ENABLE_TRACE           (1),
        .ENABLE_LOOP           (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID           (1),
        .ENABLE_NSAID           (1),
        .ENABLE_POISON          (1),
        .ENABLE_LOCK           (1),
        .SKID_DEPTH_AW          (2),
        .SKID_DEPTH_W           (4),
        .SKID_DEPTH_B           (2)
    ) u_axil5_master_wr (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // fub_aw
        .fub_awaddr              (fub_awaddr),
        // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
        // m_axil_aw
        .m_axil_awaddr           (m_axil_awaddr),
        // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
        // fub_w
        .fub_wdata               (fub_wdata),
        // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
        // m_axil_w
        .m_axil_wdata            (m_axil_wdata),
        // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
        // m_axil_b
        .m_axil_bresp            (m_axil_bresp),
        // ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
        // fub_b
        .fub_bresp               (fub_bresp),
        // ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
        // status
        .busy                    (busy)
    );

```

6.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by

running AXI5-Lite BFM's against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The `*Size` parameters and the `AW/ DW` aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

6.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr_cg](#)
 - [axil5_master_wr_mon](#)
 - `axil4_master_wr` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

6.9 Testing

`val/amba/test_axil5_master_wr.py` drives this module with the AXI5-Lite BFM's and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

6.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

7 AXI5 Master Write, Clock-Gated

Module: axil5_master_wr_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_master_wr_cg

7.1 Overview

The AXI5 Master Write, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_master_wr_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

7.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	

Parameter	Type	Default	Description
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

7.3 Ports

Your logic drives the fub_* side; the m_axil_* side faces the bus. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	
fub_awmeci	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmeci	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
fub_wdata	Input	[DW-1:0]	

Port	Direction	Width	Description
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

7.4 Functional Description

7.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

7.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH` = `AXIL_DATA_WIDTH` = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

7.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

7.5 Timing Characteristics

7.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

7.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

7.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W         (4),
    .SKID_DEPTH_B         (2),
    .CG_IDLE_COUNT_WIDTH  (4)
) u_axil5_master_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // configuration
    .cfg_cg_enable        (cfg_cg_enable),
    .cfg_cg_idle_count    (cfg_cg_idle_count),
    // fub_aw
    .fub_awaddr           (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
    // m_axil_aw
    .m_axil_awaddr        (m_axil_awaddr),
    // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
    // fub_w
    .fub_wdata            (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // m_axil_w
    .m_axil_wdata          (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bresp         (m_axil_bresp),
```

```

// ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
// fub_b
.fub_bresp          (fub_bresp),
// ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
// status
.cg_gating          (cg_gating),
.cg_idle            (cg_idle)
);

```

7.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module’s busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

7.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_mon](#)
 - [axil4_master_wr_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

7.9 Testing

`val/amba/test_axil5_master_wr_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

7.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

8 AXIL5 Master Write Monitor

Module: `axil5_master_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon`

8.1 Overview

The AXIL5 Master Write Monitor provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

8.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	

Parameter	Type	Default	Description
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when

Parameter	Type	Default	Description
			ID_FILTER_ENABLE =1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	

Parameter	Type	Default	Description
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets

Parameter	Type	Default	Description
			cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

8.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_awaddr	Input	[AW-1:0]	
fub_axil_awprot	Input	[2:0]	
fub_axil_awlock	Input	1	
fub_axil_awuser	Input	[UW-1:0]	
fub_axil_awtrace	Input	1	
fub_axil_awloop	Input	[LW-1:0]	
fub_axil_awmpam	Input	[MW-1:0]	
fub_axil_awmecid	Input	[EW-1:0]	
fub_axil_awnsaid	Input	[NW-1:0]	
fub_axil_awvalid	Input	1	
fub_axil_awready	Output	1	
fub_axil_wdata	Input	[DW-1:0]	

Port	Direction	Width	Description
fub_axil_wstrb	Input	[DW/8-1:0]	
fub_axil_wuser	Input	[UW-1:0]	
fub_axil_wpoison	Input	[PW-1:0]	
fub_axil_wvalid	Input	1	
fub_axil_wready	Output	1	
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmeced	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	

Port	Direction	Width	Description
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types

Port	Direction	Width	Description
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module's primary

Port	Direction	Width	Description
			output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction

Port	Direction	Width	Description
			count
debug_block_read_y	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

8.4 Functional Description

8.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

8.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

8.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

8.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

8.6 Timing Characteristics

8.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

8.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

8.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES      (0),
    .UNIT_ID             (8'h01),
    .AGENT_ID            (16'h000B),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING    (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC  (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC  (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC   (1'b1),
    .ENABLE_DEBUG_LOGIC  (1'b0),
    .USER_WIDTH          (4),
    .LOOP_WIDTH          (3),
    .MPAM_WIDTH          (11),
    .MECID_WIDTH         (16),
    .NSAID_WIDTH         (4),
```

```

        .ENABLE_USER            (1),
        .ENABLE_TRACE          (1),
        .ENABLE_LOOP           (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID          (1),
        .ENABLE_NSaid          (1),
        .ENABLE_POISON         (1),
        .ENABLE_LOCK           (1)
    ) u_axil5_master_wr_mon (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // status
        .cam_clear               (cam_clear),
        // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
        // m_axil_aw
        .m_axil_awaddr           (m_axil_awaddr),
        // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
        // m_axil_w
        .m_axil_wdata            (m_axil_wdata),
        // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
        // m_axil_b
        .m_axil_bresp            (m_axil_bresp),
        // ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
        // configuration
        .cfg_monitor_enable      (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
        // monitor bus
        .monbus_valid            (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

8.8 Related Modules

- [axil5_master_rd](#)
- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_rd_mon_cg](#)
- [axil5_master_wr](#)
- [axil5_master_wr_cg](#)
- [axil4_master_wr_mon](#) – the AXI4-Lite counterpart

- [AXI4-Lite modules](#)
-

8.9 Testing

`val/amba/test_axil5_master_wr_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

8.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

9 AXI5 Master Write Monitor, Clock-Gated

Module: `axil5_master_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon_cg`

9.1 Overview

The AXI5 Master Write Monitor, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon_cg` with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

9.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axil5_master_wr_monlite_cg`: the same gating logic around `axil5_master_wr_monlite` instead of `axil5_master_wr_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor

Parameter	Type	Default	Description
			in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENAB LE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_ Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STA GE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOG IC	bit	1'b1	
ENABLE_TIMEOUT_L OGIC	bit	1'b1	
ENABLE_COMPL_LOG IC	bit	1'b1	
ENABLE_THRESHOLD _LOGIC	bit	1'b1	
ENABLE_PERF_LOGI C	bit	1'b1	
ENABLE_DEBUG_LOG IC	bit	1'b0	
CG_IDLE_COUNT_WI DTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	

Parameter	Type	Default	Description
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h000B	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h01	Unit identifier

Parameter	Type	Default	Description
			emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

9.3 Ports

Your logic drives the fub_axil_* side; the m_axil_* side faces the bus. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_awaddr	Input	[AW-1:0]	
fub_axil_awprot	Input	[2:0]	
fub_axil_awlock	Input	1	
fub_axil_awuser	Input	[UW-1:0]	
fub_axil_awtrace	Input	1	

Port	Direction	Width	Description
fub_axil_awloop	Input	[LW-1:0]	
fub_axil_awmpam	Input	[MW-1:0]	
fub_axil_awmecid	Input	[EW-1:0]	
fub_axil_awnsaid	Input	[NW-1:0]	
fub_axil_awvalid	Input	1	
fub_axil_awready	Output	1	
fub_axil_wdata	Input	[DW-1:0]	
fub_axil_wstrb	Input	[DW/8-1:0]	
fub_axil_wuser	Input	[UW-1:0]	
fub_axil_wpoison	Input	[PW-1:0]	
fub_axil_wvalid	Input	1	
fub_axil_wready	Output	1	
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	

Port	Direction	Width	Description
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name

Port	Direction	Width	Description
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES	

Port	Direction	Width	Description
nable		> 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.

Port	Direction	Width	Description
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	

Port	Direction	Width	Description
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

9.4 Functional Description

9.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

9.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

9.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group’s output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

9.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

9.6 Timing Characteristics

9.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

9.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

9.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_master_wr_mon_cg #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USE_MONITOR        (1'b1),
    .N_ADDR_RANGES      (0),
    .ADDR_FILTER_ENABLE (1'b0),
```

```

.ACLK_MHZ          (100),
.USE_WDATA_ORDER_Q (1'b0),
.NUM_BANKS         (1),
.UNIT_ID           (8'h01),
.AGENT_ID           (16'h000B),
.MAX_TRANSACTIONS  (8),
.ENABLE_FILTERING   (1),
.ADD_PIPELINE_STAGE (0),
.ENABLE_ERROR_LOGIC (1'b1),
.ENABLE_TIMEOUT_LOGIC(1'b1),
.ENABLE_COMPL_LOGIC (1'b1),
.ENABLE_THRESHOLD_LOGIC(1'b1),
.ENABLE_PERF_LOGIC  (1'b1),
.ENABLE_DEBUG_LOGIC (1'b0),
.CG_IDLE_COUNT_WIDTH (4),
.USER_WIDTH         (4),
.LOOP_WIDTH         (3),
.MPAM_WIDTH         (11),
.MECID_WIDTH        (16),
.NSAID_WIDTH        (4),
.ENABLE_USER        (1),
.ENABLE_TRACE       (1),
.ENABLE_LOOP        (1),
.ENABLE_MPAM        (1),
.ENABLE_MECID       (1),
.ENABLE_NSAID       (1),
.ENABLE_POISON      (1),
.ENABLE_LOCK        (1)
) u_axil5_master_wr_mon_cg (
    // clock/reset
    .aclk          (aclk),
    .aresetn       (aresetn),
    // status
    .cam_clear     (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
    // m_axil_aw
    .m_axil_awaddr (m_axil_awaddr),
    // ... `m_axil_awprot`, `m_axil_awlock`, `m_axil_awuser`, +7 more
    // m_axil_w
    .m_axil_wdata  (m_axil_wdata),
    // ... `m_axil_wstrb`, `m_axil_wuser`, `m_axil_wpoison`, +2 more
    // m_axil_b
    .m_axil_bresp  (m_axil_bresp),
    // ... `m_axil_buser`, `m_axil_btrace`, `m_axil_bloop`, +2 more
    // configuration
    .cfg_monitor_enable (cfg_monitor_enable),
    // ... `cfg_error_enable`, `cfg_timeout_enable`,

```

```
`cfg_perf_enable`, +30 more
    // monitor bus
    .monbus_valid      (monbus_valid)
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

9.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_master_wr_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

9.9 Testing

`val/amba/test_axil5_master_wr_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

9.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

10 AXIL5 Slave Read

Module: axil5_slave_rd.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_rd

10.1 Overview

The AXIL5 Slave Read provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_rd with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
-

10.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	$(DW / 64) > 0$? $(DW / 64) :$ 1	
ARSize	int	$AW + 3 +$	
RSize	int	$DW + 2 +$	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

10.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
s_axil_araddr	Input	[AW-1:0]	

Port	Direction	Width	Description
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arloop	Input	[LW-1:0]	
s_axil_arpam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
fub_araddr	Output	[AW-1:0]	
fub_arprot	Output	[2:0]	
fub_arlock	Output	1	
fub_aruser	Output	[UW-1:0]	
fub_artrace	Output	1	
fub_arloop	Output	[LW-1:0]	
fub_arpam	Output	[MW-1:0]	
fub_armecid	Output	[EW-1:0]	
fub_arnsaid	Output	[NW-1:0]	
fub_arvalid	Output	1	
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	

Port	Direction	Width	Description
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
busy	Output	1	

10.4 Functional Description

10.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

10.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

10.4.3 It transports; it does not interpret

MPAM, MECID, NSAIID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

10.5 Timing Characteristics

10.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

10.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

10.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AR        (2),
    .SKID_DEPTH_R         (4)
) u_axil5_slave_rd (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // s_axil_ar
    .s_axil_araddr        (s_axil_araddr),
```

```

// ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
// fub_ar
.fub_araddr      (fub_araddr),
// ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
// fub_r
.fub_rdata       (fub_rdata),
// ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
// s_axil_r
.s_axil_rdata    (s_axil_rdata),
// ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
// status
.busy            (busy)
);

```

10.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/ASize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

10.8 Related Modules

- [axil5_master_rd](#)
- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_rd_mon_cg](#)
- [axil5_master_wr](#)
- [axil5_master_wr_cg](#)

- `axil4_slave_rd` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

10.9 Testing

`val/amba/test_axil5_slave_rd.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

10.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

11 AXI5 Slave Read, Clock-Gated

Module: axil5_slave_rd_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_rd_cg

11.1 Overview

The AXI5 Slave Read, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_rd_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

11.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

11.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	

Port	Direction	Width	Description
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arloop	Input	[LW-1:0]	
s_axil_arpam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
fub_araddr	Output	[AW-1:0]	
fub_arprot	Output	[2:0]	
fub_arlock	Output	1	
fub_aruser	Output	[UW-1:0]	
fub_artrace	Output	1	
fub_arloop	Output	[LW-1:0]	
fub_arpam	Output	[MW-1:0]	
fub_armecid	Output	[EW-1:0]	
fub_arnsaid	Output	[NW-1:0]	
fub_arvalid	Output	1	
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	

Port	Direction	Width	Description
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

11.4 Functional Description

11.4.1 Optional signal groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP,	LOOP_WIDTH

Group	Parameter	Signals on this module	Width
		RLOOP	
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

11.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

11.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

11.5 Timing Characteristics

11.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

11.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

11.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
```

```

        .ENABLE_LOOP            (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID          (1),
        .ENABLE_NSAID          (1),
        .ENABLE_POISON         (1),
        .ENABLE_LOCK            (1),
        .SKID_DEPTH_AR         (2),
        .SKID_DEPTH_R          (4),
        .CG_IDLE_COUNT_WIDTH   (4)
    ) u_axil5_slave_rd_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // configuration
        .cfg_cg_enable           (cfg_cg_enable),
        .cfg_cg_idle_count       (cfg_cg_idle_count),
        // s_axil_ar
        .s_axil_araddr           (s_axil_araddr),
        // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
        // fub_ar
        .fub_araddr              (fub_araddr),
        // ... `fub_arprot`, `fub_arlock`, `fub_aruser`, +7 more
        // fub_r
        .fub_rdata                (fub_rdata),
        // ... `fub_rresp`, `fub_ruser`, `fub_rtrace`, +4 more
        // s_axil_r
        .s_axil_rdata            (s_axil_rdata),
        // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
        // status
        .cg_gating                (cg_gating),
        .cg_idle                  (cg_idle)
    );

```

11.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating` – see `vault/handbook/design/clock-gating-activity-terms.md`.

busy is not brought out here. The plain `_cg` wrappers consume the base module's busy internally as a wake term; use `cg_idle` or `cg_gating` for idle detection. The `_mon_cg` variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only. With it low the clock free-runs and the module behaves exactly like its base.

11.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_rd_cg` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

11.9 Testing

`val/amba/test_axil5_slave_rd_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

11.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

12 AXIL5 Slave Read Monitor

Module: `axil5_slave_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon`

12.1 Overview

The AXIL5 Slave Read Monitor provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon` with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

12.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE

Parameter	Type	Default	Description
			=1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told

Parameter	Type	Default	Description
			apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

12.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_axil_* side. The cfg_* and monbus_* ports belong to the monitor.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arloop	Input	[LW-1:0]	
s_axil_armspam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	

Port	Direction	Width	Description
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_arpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask

Port	Direction	Width	Description
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1 : 0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the

Port	Direction	Width	Description
			group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force	Input	1	

Port	Direction	Width	Description
_close			
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

12.4 Functional Description

12.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

12.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

12.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

12.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.

axi_monitor_trans_mgr fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets ADDR_FILTER_ENABLE but leaves cfg_addr_filter_enable low filters nothing and looks broken. The parameter and the runtime port are both required.

12.6 Timing Characteristics

12.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one gaxi_skid_buffer. That module registers both rd_valid and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

12.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, ASize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

12.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R      (4),
```

```

.AXIL_ADDR_WIDTH      (32),
.AXIL_DATA_WIDTH      (32),
.ACLK_MHZ             (100),
.USE_MONITOR          (1'b1),
.N_ADDR_RANGES        (0),
.UNIT_ID              (8'h02),
.AGENT_ID              (16'h0014),
.MAX_TRANSACTIONS     (8),
.USE_WDATA_ORDER_Q    (1'b0),
.NUM_BANKS            (1),
.ID_FILTER_ENABLE     (1'b0),
.ADDR_FILTER_ENABLE   (1'b0),
.ID_MATCH_BASE        (0),
.ID_MATCH_COUNT       (0),
.ENABLE_FILTERING     (1),
.ADD_PIPELINE_STAGE   (0),
.ENABLE_ERROR_LOGIC   (1'b1),
.ENABLE_TIMEOUT_LOGIC (1'b1),
.ENABLE_COMPL_LOGIC   (1'b1),
.ENABLE_THRESHOLD_LOGIC (1'b1),
.ENABLE_PERF_LOGIC    (1'b1),
.ENABLE_DEBUG_LOGIC   (1'b0),
.USER_WIDTH           (4),
.LOOP_WIDTH           (3),
.MPAM_WIDTH           (11),
.MECID_WIDTH          (16),
.NSAID_WIDTH          (4),
.ENABLE_USER          (1),
.ENABLE_TRACE         (1),
.ENABLE_LOOP          (1),
.ENABLE_MPAM          (1),
.ENABLE_MECID         (1),
.ENABLE_NSAID         (1),
.ENABLE_POISON        (1),
.ENABLE_LOCK          (1)
) u_axil5_slave_rd_mon (
    // clock/reset
    .aclk                (aclk),
    .aresetn             (aresetn),
    // status
    .cam_clear           (cam_clear),
    // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+31 more
    // s_axil_ar
    .s_axil_araddr       (s_axil_araddr),
    // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
    // s_axil_r
    .s_axil_rdata        (s_axil_rdata),

```

```
// ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
// configuration
.cfg_monitor_enable (cfg_monitor_enable),
// ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
// monitor bus
.monbus_valid (monbus_valid)
// ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
);
```

12.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_rd_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

12.9 Testing

`val/amba/test_axil5_slave_rd_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – TBClasses/axil5 sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

12.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

13 AXI5 Slave Read Monitor, Clock-Gated

Module: axil5_slave_rd_mon_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_rd_mon_cg

13.1 Overview

The AXI5 Slave Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_rd_mon_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite read path (AR and R channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

13.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as axil5_slave_rd_monlite_cg: the same gating logic around axil5_slave_rd_monlite instead of axil5_slave_rd_mon, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon;

Parameter	Type	Default	Description
N_ADDR_RANGES	int	0	outputs tied 0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	

Parameter	Type	Default	Description
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0014	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier

Parameter	Type	Default	Description
			emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

13.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_axil_* side. The cfg_* and monbus_* ports belong to the monitor, and cfg_cg_* runs the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	

Port	Direction	Width	Description
s_axil_arloop	Input	[LW-1:0]	
s_axil_arpam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_arpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	

Port	Direction	Width	Description
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error

Port	Direction	Width	Description
			event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1 : 0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready

Port	Direction	Width	Description
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions

Port	Direction	Width	Description
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

13.4 Functional Description

13.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

13.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

13.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

13.5 Design Notes

Four things worth knowing before you trust what comes out of the monitor:

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1.
`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

13.6 Timing Characteristics

13.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AR	2 entries	Skid depth on the AR channel
SKID_DEPTH_R	4 entries	Skid depth on the R channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

13.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, AWSize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

13.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_rd_mon_cg #(  
    .SKID_DEPTH_AR      (2),  
    .SKID_DEPTH_R       (4),  
    .AXIL_ADDR_WIDTH    (32),  
    .AXIL_DATA_WIDTH    (32),  
    .USE_MONITOR        (1'b1),  
    .N_ADDR_RANGES      (0),  
    .ADDR_FILTER_ENABLE (1'b0),  
    .ACLK_MHZ           (100),  
    .USE_WDATA_ORDER_Q  (1'b0),  
    .NUM_BANKS          (1),  
    .UNIT_ID            (8'h02),  
    .AGENT_ID           (16'h0014),  
    .MAX_TRANSACTIONS   (8),  
    .ENABLE_FILTERING   (1),  
    .ADD_PIPELINE_STAGE (0),  
    .ENABLE_ERROR_LOGIC (1'b1),  
    .ENABLE_TIMEOUT_LOGIC(1'b1),  
    .ENABLE_COMPL_LOGIC (1'b1),  
    .ENABLE_THRESHOLD_LOGIC(1'b1),  
    .ENABLE_PERF_LOGIC  (1'b1),  
    .ENABLE_DEBUG_LOGIC (1'b0),  
    .CG_IDLE_COUNT_WIDTH (4),  
    .USER_WIDTH         (4),  
    .LOOP_WIDTH         (3),  
    .MPAM_WIDTH         (11),  
    .MECID_WIDTH        (16),  
    .NSAID_WIDTH        (4),  
    .ENABLE_USER        (1),  
    .ENABLE_TRACE       (1),  
    .ENABLE_LOOP        (1),  
)
```

```

        .ENABLE_MPAM            (1),
        .ENABLE_MECID          (1),
        .ENABLE_NSAID          (1),
        .ENABLE_POISON          (1),
        .ENABLE_LOCK            (1)
    ) u_axil5_slave_rd_mon_cg (
        // clock/reset
        .aclk                    (aclk),
        .aresetn                 (aresetn),
        // status
        .cam_clear               (cam_clear),
        // ... `fub_axil_araddr`, `fub_axil_arprot`, `fub_axil_arlock`,
+32 more
        // s_axil_ar
        .s_axil_araddr           (s_axil_araddr),
        // ... `s_axil_arprot`, `s_axil_arlock`, `s_axil_aruser`, +7 more
        // s_axil_r
        .s_axil_rdata            (s_axil_rdata),
        // ... `s_axil_rresp`, `s_axil_ruser`, `s_axil_rtrace`, +4 more
        // configuration
        .cfg_monitor_enable      (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +30 more
        // monitor bus
        .monbus_valid            (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

13.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_rd_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

13.9 Testing

`val/amba/test_axil5_slave_rd_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

13.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

14 AXIL5 Slave Write

Module: axil5_slave_wr.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_wr

14.1 Overview

The AXIL5 Slave Write provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_wr with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
-

14.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	
BSize	int	2 +	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

14.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side.

Port	Direction	Width	Description
aclk	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awnecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awlock	Output	1	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awloop	Output	[LW-1:0]	
fub_awmpam	Output	[MW-1:0]	
fub_awnecid	Output	[EW-1:0]	
fub_awnsaid	Output	[NW-1:0]	
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	

Port	Direction	Width	Description
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
busy	Output	1	

14.4 Functional Description

14.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER,	USER_WIDTH

Group	Parameter	Signals on this module	Width
		BUSER, RUSER	
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

14.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM's: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

14.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

14.5 Timing Characteristics

14.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

14.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

14.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```

axil5_slave_wr #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W          (4),
    .SKID_DEPTH_B         (2)
) u_axil5_slave_wr (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // s_axil_aw
    .s_axil_awaddr        (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // fub_aw
    .fub_awaddr           (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
    // s_axil_w
    .s_axil_wdata          (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // fub_w
    .fub_wdata            (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // fub_b
    .fub_bresp            (fub_bresp),
    // ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
    // s_axil_b
    .s_axil_bresp          (s_axil_bresp),
    // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
    // status
    .busy                  (busy)
);

```

14.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what `val/amba/test_axil5_axil4_equivalence.py` asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. `gaxi_skid_buffer` registers both `rd_valid` and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

14.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - `axil4_slave_wr` – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

14.9 Testing

`val/amba/test_axil5_slave_wr.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

14.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

15 AXIL5 Slave Write, Clock-Gated

Module: axil5_slave_wr_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_wr_cg

15.1 Overview

The AXIL5 Slave Write, Clock-Gated provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_wr_cg with those groups threaded through the packed SKID payload.

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - One amba_clock_gate_ctrl gating the whole inner module
-

15.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	

Parameter	Type	Default	Description
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

15.3 Ports

The s_axil_* side faces the bus; your backend drives the fub_* side. The cfg_cg_* inputs run the clock gate.

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmeci	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awlock	Output	1	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awloop	Output	[LW-1:0]	
fub_awmpam	Output	[MW-1:0]	
fub_awmeci	Output	[EW-1:0]	
fub_awnsaid	Output	[NW-1:0]	
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	

Port	Direction	Width	Description
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

15.4 Functional Description

15.4.1 Optional signal groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

15.4.2 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH` = `AXIL_DATA_WIDTH` = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

15.4.3 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried – never generated, never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's output is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

15.5 Timing Characteristics

15.5.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	4 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the unstalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

15.5.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

15.6 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_cg #(
    .AXIL_ADDR_WIDTH      (32),
    .AXIL_DATA_WIDTH      (32),
    .USER_WIDTH           (4),
    .LOOP_WIDTH           (3),
    .MPAM_WIDTH           (11),
    .MECID_WIDTH          (16),
    .NSAID_WIDTH          (4),
    .ENABLE_USER          (1),
    .ENABLE_TRACE         (1),
    .ENABLE_LOOP          (1),
    .ENABLE_MPAM          (1),
    .ENABLE_MECID         (1),
    .ENABLE_NSAID         (1),
    .ENABLE_POISON        (1),
    .ENABLE_LOCK          (1),
    .SKID_DEPTH_AW        (2),
    .SKID_DEPTH_W         (4),
    .SKID_DEPTH_B         (2),
    .CG_IDLE_COUNT_WIDTH  (4)
) u_axil5_slave_wr_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // configuration
    .cfg_cg_enable        (cfg_cg_enable),
    .cfg_cg_idle_count    (cfg_cg_idle_count),
    // s_axil_aw
    .s_axil_awaddr        (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // fub_aw
    .fub_awaddr          (fub_awaddr),
    // ... `fub_awprot`, `fub_awlock`, `fub_awuser`, +7 more
    // s_axil_w
    .s_axil_wdata         (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // fub_w
    .fub_wdata           (fub_wdata),
    // ... `fub_wstrb`, `fub_wuser`, `fub_wpoison`, +2 more
    // fub_b
    .fub_bresp           (fub_bresp),
```

```

// ... `fub_buser`, `fub_btrace`, `fub_bloop`, +2 more
// s_axil_b
.s_axil_bresp          (s_axil_bresp),
// ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
// status
.cg_gating             (cg_gating),
.cg_idle               (cg_idle)
);

```

15.7 Design Notes

All optional groups off is AXI4-Lite. ARSize/AWSize/WSize/RSize/ BSize are conditional sums over the ENABLE_* parameters. Disable every group and they collapse to the AXI4-Lite payload widths exactly, which is what val/amba/test_axil5_axil4_equivalence.py asserts by running AXI5-Lite BFM against the AXI4-Lite RTL. If that test ever fails, AXI5-Lite has drifted from the protocol it extends.

Do not override a derived parameter. The *Size parameters and the AW/ DW aliases are declared as parameter so the elaborator can compute them, not so callers can set them. Forcing one to the AXI4-Lite value while a group is enabled part-selects past the end of the vector – the failure reads “Extracting 4 bits from only 2 bit number”, and it is how this family’s test port broke.

No transaction IDs. AXI4-Lite and AXI5-Lite have none, so anything ID-shaped in the shared monitor core is inert here and the wrappers tie the ID inputs to zero.

The skid buffer is not a bypass. gaxi_skid_buffer registers both rd_valid and its storage, so every transfer costs one cycle even unstalled. Depth buys backpressure absorption, not throughput.

A peer’s READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into user_valid pins this block permanently awake and defeats gating entirely, silently, because function is unaffected. This wrapper wakes on VALIDs and pending work only. val/amba/test_cg_peer_ready.py parks the peer READY high, holds every VALID low, and requires cg_gating – see vault/handbook/design/clock-gating-activity-terms.md.

busy is not brought out here. The plain _cg wrappers consume the base module’s busy internally as a wake term; use cg_idle or cg_gating for idle detection. The _mon_cg variants do expose busy.

cfg_cg_enable is not a kill switch. It arms gating and reaches amba_clock_gate_ctrl only. With it low the clock free-runs and the module behaves exactly like its base.

15.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_wr_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

15.9 Testing

`val/amba/test_axil5_slave_wr_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically.

15.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

16 AXIL5 Slave Write Monitor

Module: `axil5_slave_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon`

16.1 Overview

The AXIL5 Slave Write Monitor gives a slave device a buffered AXI5-Lite write interface — and keeps a transaction monitor on the path while it's at it.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon` with those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

16.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
-

16.2 Parameters

A lite-monitor version of this wrapper exists as `axil5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
<code>SKID_DEPTH_AW</code>	<code>int</code>	2	
<code>SKID_DEPTH_W</code>	<code>int</code>	2	
<code>SKID_DEPTH_B</code>	<code>int</code>	2	
<code>AXIL_ADDR_WIDTH</code>	<code>int</code>	32	

Parameter	Type	Default	Description
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	Per-instance ID-slice filter, inherited from the shared monitor core. Leave at 0 on AXI4-Lite. The wrapper hardwires cmd_id/data_id/resp_id to 1'b0, so enabling this with ID_MATCH_BASE above 0 makes id_owned(0) false for every transaction and drops ALL monitoring.
ADDR_FILTER_ENAB	bit	1'b0	

Parameter	Type	Default	Description
LE			
ID_MATCH_BASE	int	0	First ID this instance owns when ID_FILTER_ENABLE =1. Must stay 0 on AXI4-Lite – every transaction reports ID 0.
ID_MATCH_COUNT	int	0	Number of IDs owned from ID_MATCH_BASE. 0 = all IDs (the filter passes everything).
ACTIVE_TRANS_THR ESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	

Parameter	Type	Default	Description
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-bit Agent ID for monitor packets)
UNIT_ID		8'h02	Unit identifier emitted in the unit_id field of every monitor packet. Give each

Parameter	Type	Default	Description
			monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

16.3 Ports

Port	Direction	Width	Description
aclk	Input	1	sync clear of the monitor trans CAM
aresetn	Input	1	
cam_clear	Input	1	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	

Port	Direction	Width	Description
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awlock	Output	1	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	
fub_axil_awloop	Output	[LW-1:0]	
fub_axil_awmpam	Output	[MW-1:0]	
fub_axil_awmecid	Output	[EW-1:0]	
fub_axil_awnsaid	Output	[NW-1:0]	
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bresp	Input	[1:0]	

Port	Direction	Width	Description
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bloop	Input	[LW-1:0]	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for

Port	Direction	Width	Description
			packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the

Port	Direction	Width	Description
			module's primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count

Port	Direction	Width	Description
transaction_count	Output	[31:0]	Total transaction count
debug_block_read_y	Output	1	Configuration conflict detected
cfg_conflict_error	Output	1	
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

16.4 Functional Description

AXI5-Lite adds eight optional signal groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH

Group	Parameter	Signals on this module	Width
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

16.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence isn't academic. It's what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

16.4.2 It transports; it does not interpret

This module is a pipe, not a policy engine. MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

One kindness to integrators: a disabled group's OUTPUT is driven to zero rather than left dangling, so if you disable a group downstream of a block that enables it, you see a defined value instead of X.

16.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It doesn't check MPAM/MECID/NSAID consistency, and it doesn't validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

16.6 Timing Characteristics

16.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the uninstalled case** – there is no combinational bypass from the upstream payload to the downstream

one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

16.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: ARSize, ASize, WSize, RSize and BSize are conditional sums over the ENABLE_* parameters, so disabling a group narrows the storage without changing latency.

16.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH     (32),
    .ACLK_MHZ           (100),
    .USE_MONITOR         (1'b1),
    .N_ADDR_RANGES      (0),
    .UNIT_ID             (8'h02),
    .AGENT_ID            (16'h0015),
    .MAX_TRANSACTIONS    (8),
    .USE_WDATA_ORDER_Q   (1'b0),
    .NUM_BANKS           (1),
    .ID_FILTER_ENABLE    (1'b0),
    .ADDR_FILTER_ENABLE  (1'b0),
    .ID_MATCH_BASE       (0),
    .ID_MATCH_COUNT      (0),
    .ENABLE_FILTERING     (1),
    .ADD_PIPELINE_STAGE  (0),
    .ENABLE_ERROR_LOGIC   (1'b1),
    .ENABLE_TIMEOUT_LOGIC(1'b1),
    .ENABLE_COMPL_LOGIC   (1'b1),
    .ENABLE_THRESHOLD_LOGIC(1'b1),
    .ENABLE_PERF_LOGIC    (1'b1),
    .ENABLE_DEBUG_LOGIC   (1'b0),
    .USER_WIDTH           (4),
```

```

        .LOOP_WIDTH           (3),
        .MPAM_WIDTH           (11),
        .MECID_WIDTH          (16),
        .NSAID_WIDTH          (4),
        .ENABLE_USER          (1),
        .ENABLE_TRACE         (1),
        .ENABLE_LOOP          (1),
        .ENABLE_MPAM          (1),
        .ENABLE_MECID         (1),
        .ENABLE_NSAID         (1),
        .ENABLE_POISON        (1),
        .ENABLE_LOCK          (1)
    ) u_axil5_slave_wr_mon (
        // clock/reset
        .aclk                   (aclk),
        .aresetn                (aresetn),
        // status
        .cam_clear              (cam_clear),
        // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+35 more
        // s_axil_aw
        .s_axil_awaddr          (s_axil_awaddr),
        // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
        // s_axil_w
        .s_axil_wdata           (s_axil_wdata),
        // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
        // s_axil_b
        .s_axil_bresp           (s_axil_bresp),
        // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
        // configuration
        .cfg_monitor_enable     (cfg_monitor_enable),
        // ... `cfg_error_enable`, `cfg_timeout_enable`,
`cfg_perf_enable`, +28 more
        // monitor bus
        .monbus_valid           (monbus_valid)
        // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`
    );

```

16.8 Related Modules

- [axil5_master_rd](#)
- [axil5_master_rd_cg](#)
- [axil5_master_rd_mon](#)
- [axil5_master_rd_mon_cg](#)

- [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_wr_mon](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

16.9 Testing

`val/amba/test_axil5_slave_wr_mon.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

16.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)

17 AXI5 Slave Write Monitor, Clock-Gated

Module: axil5_slave_wr_mon_cg.sv **Location:** rtl/amba/axil5/ **AXI4-Lite counterpart:** axil4_slave_wr_mon_cg

17.1 Overview

The AXI5 Slave Write Monitor, Clock-Gated gives a slave device a buffered AXI5-Lite write interface — the same monitor as the plain variant, with clock gating stacked on top and the ready signals held low while gated.

If you take one thing from this page, take this: AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally axil4_slave_wr_mon_cg with those groups threaded through the packed SKID payload. Know the AXI4-Lite part and you know this one.

17.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
 - Single-beat transactions: AXI-Lite has no burst support
 - Configurable skid buffers on every channel for timing closure
 - Eight optional signal groups, each independently enabled
 - Bit-identical to the AXI4-Lite module of the same name with all groups disabled
 - Integrated transaction monitor emitting monbus packets
 - Monitor plus clock gating; ready signals held low while gated
-

17.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as axil5_slave_wr_monlite_cg: the same gating logic around axil5_slave_wr_monlite instead of axil5_slave_wr_mon, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AGENT_ID		16'h0015	
			Agent identifier emitted in the agent_id field of every monitor packet. Pairs with UNIT_ID to identify the packet source. (16-

Parameter	Type	Default	Description
UNIT_ID		8'h02	bit Agent ID for monitor packets) Unit identifier emitted in the unit_id field of every monitor packet. Give each monitored interface a distinct value or the packets cannot be told apart at the collector. (8-bit Unit ID for monitor packets)

The derived parameters (AW, DW, UW, ... and the *Size payload widths) are computed from the ones above. **Do not override them.** Force a *Size to a value the RTL didn't derive and every optional-group field turns into a part-select past the end of the vector — which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

17.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	

Port	Direction	Width	Description
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awlock	Output	1	
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	
fub_axil_awloop	Output	[LW-1:0]	
fub_axil_awmpam	Output	[MW-1:0]	
fub_axil_awmecid	Output	[EW-1:0]	
fub_axil_awnsaid	Output	[NW-1:0]	
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	

Port	Direction	Width	Description
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bloop	Input	[LW-1:0]	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name

Port	Direction	Width	Description
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES	

Port	Direction	Width	Description
nable		$> 0 ?$ N_ADDR_RANGES : 1) - 1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
monbus_packet	Output	monitor_packet_t (128)	The monitor packet itself – the module’s primary output. Valid when monbus_valid is high.
monbus_timestamp	Output	monbus_timestamp_t	Side-band sampled time for the packet on monbus_packet.
i_mon_time	Input	monbus_timestamp_t	Shared free-running timestamp, driven from the group/aggregator so every monitor stamps against one clock.
cfg_addr_range_low	Input	[AW-1:0] x N_ADDR_RANGES	Low bound of each address-range comparator; only present when N_ADDR_RANGES > 0.

Port	Direction	Width	Description
cfg_addr_range_high	Input	[AW-1:0] x N_ADDR_RANGES	High bound of each address-range comparator; pairs with cfg_addr_range_low.
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	

Port	Direction	Width	Description
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

17.4 Functional Description

AXI5-Lite adds eight optional signal groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when it's enabled — turn it off and those bits simply aren't in the vector.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

17.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence isn't academic. It's what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFM: with no groups enabled an AXI5-Lite interface is an AXI4-Lite interface, so the same testbench binds to either.

17.4.2 It transports; it does not interpret

This module is a pipe, not a policy engine. MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

One kindness to integrators: a disabled group's OUTPUT is driven to zero rather than left dangling, so if you disable a group downstream of a block that enables it, you see a defined value instead of X.

17.5 Design Notes

A handful of traps specific to the monitored variants. Each one is written down because someone hit it.

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It doesn't check MPAM/MECID/NSAID consistency, and it doesn't validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong — silently. That's the worst kind of wrong.

NUM_BANKS > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`.

`axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination. At least that one fails loud.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

17.6 Timing Characteristics

17.6.1 Buffer Depths and Latency

Parameter	Default	Channel
SKID_DEPTH_AW	2 entries	Skid depth on the AW channel
SKID_DEPTH_W	2 entries	Skid depth on the W channel
SKID_DEPTH_B	2 entries	Skid depth on the B channel

Each channel traverses one `gaxi_skid_buffer`. That module registers both `rd_valid` and the storage array, so the **1-cycle input-to-output latency applies on every transfer, including the un stalled case** – there is no combinational bypass from the upstream payload to the downstream one. Full throughput (one transfer per cycle) is still sustained once the pipeline is primed; the depth sets how much backpressure can be absorbed before it propagates upstream, not the steady-state rate.

Legal depth range is 2..8 inclusive, odd values included.

17.6.2 Optional-group effect

The AXI5-Lite optional groups widen the packed skid payload but do not add a pipeline stage: `ARSize`, `AWSize`, `WSize`, `RSize` and `BSize` are conditional sums over the `ENABLE_*` parameters, so disabling a group narrows the storage without changing latency.

17.7 Usage Examples

Every parameter and port below is taken from the module declaration; the elisions name the remaining members of each group, all of which appear in the Ports table above. Override only what your integration needs, and never override a derived parameter – see the note under Parameters.

```
axil5_slave_wr_mon_cg #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (2),
    .SKID_DEPTH_B       (2),
    .AXIL_ADDR_WIDTH    (32),
    .AXIL_DATA_WIDTH    (32),
    .USE_MONITOR        (1'b1),
```

```

.N_ADDR_RANGES          (0),
.ADDR_FILTER_ENABLE     (1'b0),
.ACLK_MHZ                (100),
.USE_WDATA_ORDER_Q      (1'b0),
.NUM_BANKS               (1),
.UNIT_ID                 (8'h02),
.AGENT_ID                (16'h0015),
.MAX_TRANSACTIONS        (8),
.ENABLE_FILTERING        (1),
.ADD_PIPELINE_STAGE      (0),
.ENABLE_ERROR_LOGIC      (1'b1),
.ENABLE_TIMEOUT_LOGIC    (1'b1),
.ENABLE_COMPL_LOGIC      (1'b1),
.ENABLE_THRESHOLD_LOGIC  (1'b1),
.ENABLE_PERF_LOGIC       (1'b1),
.ENABLE_DEBUG_LOGIC      (1'b0),
.CG_IDLE_COUNT_WIDTH     (4),
.USER_WIDTH              (4),
.LOOP_WIDTH              (3),
.MPAM_WIDTH              (11),
.MECID_WIDTH             (16),
.NSAID_WIDTH             (4),
.ENABLE_USER             (1),
.ENABLE_TRACE            (1),
.ENABLE_LOOP             (1),
.ENABLE_MPAM             (1),
.ENABLE_MECID            (1),
.ENABLE_NSAID            (1),
.ENABLE_POISON           (1),
.ENABLE_LOCK             (1)
) u_axil5_slave_wr_mon_cg (
    // clock/reset
    .aclk                (aclk),
    .aresetn              (aresetn),
    // status
    .cam_clear            (cam_clear),
    // ... `fub_axil_awaddr`, `fub_axil_awprot`, `fub_axil_awlock`,
+36 more
    // s_axil_aw
    .s_axil_awaddr        (s_axil_awaddr),
    // ... `s_axil_awprot`, `s_axil_awlock`, `s_axil_awuser`, +7 more
    // s_axil_w
    .s_axil_wdata          (s_axil_wdata),
    // ... `s_axil_wstrb`, `s_axil_wuser`, `s_axil_wpoison`, +2 more
    // s_axil_b
    .s_axil_bresp          (s_axil_bresp),
    // ... `s_axil_buser`, `s_axil_btrace`, `s_axil_bloop`, +2 more
    // configuration

```

```
.cfg_monitor_enable (cfg_monitor_enable),  
    // ... `cfg_error_enable`, `cfg_timeout_enable`,  
    `cfg_perf_enable`, +30 more  
    // monitor bus  
    .monbus_valid      (monbus_valid)  
    // ... `monbus_ready`, `monbus_packet`, `monbus_timestamp`  
);
```

17.8 Related Modules

- [axil5_master_rd](#)
 - [axil5_master_rd_cg](#)
 - [axil5_master_rd_mon](#)
 - [axil5_master_rd_mon_cg](#)
 - [axil5_master_wr](#)
 - [axil5_master_wr_cg](#)
 - [axil4_slave_wr_mon_cg](#) – the AXI4-Lite counterpart
 - [AXI4-Lite modules](#)
-

17.9 Testing

`val/amba/test_axil5_slave_wr_mon_cg.py` drives this module with the AXI5-Lite BFM and **every optional group enabled** — `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake — and the version you'd rather get.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition, and a fix to the AXI4-Lite flow reaches this module automatically. That's exactly how you want a variant family to work.

17.10 Navigation

[AXI5-Lite index](#) | [AMBA overview](#)